以 FPGA 實現金融微結構報價決策之確定性低延遲推論：以 PHC 為例

---

1. Meeting 報告大綱

1.1 標題與動機

- 動機起點：對量化程式交易感興趣 → 選擇金融時序訊號領域 → 好奇模型加速在此領域的效果 → 導聚聽導師說 PHC 這個模型後，發現其參數小的特點或許適合拿來 FPGA 實作。
- HFT 系統的風險窗口由最差情況延遲決定，因此比起平均吞吐，更關注尾端延遲與時序確定性
- 對照組：TCN 已有完整資料（scratch / distilled 對照 PHC teacher），直接可用；DeepLOB / FI-2010 目前僅完成資料下載（791MB zip，尚未解壓、無任何模型或結果），不建議放進本次報告的主線，移至 future work。

1.2 貢獻

- ① 權衡 PHC 模型在短期委託簿 action-value 預測上的可行性與效果——軟體回測顯示 PHC 系列在 val/NEW3/NEW4 三個 split 上排名穩定優於簡單規則與 HGB 基準（9 組合敏感度掃描 100% 一致）。
- ② 系統性的 HLS 資源優化案例——手刻 HLS 八輪迭代將 DSP 用量從初版 2636%（不可實現）降至 21%（P&R 實測 21.36%），並定位出三個具體的工具鏈陷阱根因（雙精度 tanh、PIPELINE 強制展開、全域 pipeline_loops 閾值）。
- ③ 分層驗證等級的誠實揭露——每個關鍵數字都標註其驗證等級（估計/csim/cosim/P&R 實測/CPU 對照），而非籠統宣稱「已驗證」；PHC 達到 csim→csynth→cosim→P&R timing met 全鏈驗證，TCN 目前僅到 csynth。
- ④ 工具鏈選型的失效記錄——hls4ml 在 TCN 的 ConstantPad1d 解析階段直接失敗（連 C++ 專案都無法生成），據此改採手刻 HLS，此失效記錄本身是可複用的工程知識。
- ⑤ 遞迴必要性的初步證據——TCN（feedforward、可平行化）scratch 訓練直接打平甚至略贏 PHC teacher 的 causal-fill PnL；消融實驗顯示優勢主要來自「序列順序＋閘控狀態結構」而非某個學到的具體 decay 值。這對「是否真的需要遞迴」提出初步質疑，也指出通往更低延遲（feedforward 可管線化）的路徑。

1.3 技術細節（對應 Methodology 三小節）

- ① 模型的設計方法：PHC-lite 的閘控/衰減遞迴設計、與 TCN 對照組的架構選擇
- ② 軟體驗證與回測方法：資料切分協定、causal-fill PnL 回測、蒸餾與消融實驗設計
- ③ 將模型放上 FPGA 的權衡與取捨：hls4ml vs 手刻 HLS 的選型理由、資源/延遲/精度三者的優化取捨

1.4 Future Work（按誠實度／進度排序）

- ① PYNQ-Z2 上板實測
- ② 延遲收斂路線圖——83.16μs（P&R 實測）→ 23.16μs（unroll factor=16，csynth 已驗證但未過 P&R）的既有進展，加上分析指出通往 <10μs／~1μs 需要架構層級改動（減少 dense block 數量、結構化或對角矩陣變換），而非單靠調頻或量化。

- ③ Hardware-aware 特徵選擇——與隊友協作方向，可作為跨組合作亮點。
- ④ 公開基準對照（FI-2010/DeepLOB）
- ⑤ 更嚴謹的 CPU 基準——目前 CPU baseline 是 PyTorch Python 版（float32, no_grad），未使用 LibTorch C++/TorchScript；p99 9.4× 的優勢中有部分來自 Python 層開銷（GC、排程），換用 C++ 基準會是更硬的對照組，應誠實揭露為現有比較的限制。

---

2. Methodology（草稿，可直接用於論文）

寫作原則：每小節只回答「怎麼做／為什麼這樣做」，不夾帶結果數字（數字留給 Experiments）；每個方法選擇都附一句「為什麼不用另一個做法」，這是最省字又最有說服力的寫法。

2.1 模型設計方法

PHC-lite 是 PHCSSM 論文的簡化實作：保留「可學習的閘控/衰減遞迴狀態更新」與「單一時間步內的多次訊息傳遞（within-timestep multi-transmission loop）」，捨棄 spike 機制、STDP、surrogate-gradient spiking neuron——因為目標是評估「生物啟發的極簡閘控」在 FPGA 上的可行性，而非完整複現生物神經動力學。

模型輸入：為當前特徵向量與前一時刻狀態，經 encode（linear→LayerNorm→SiLU）後，以 sigmoid 閘控與 tanh 候選值更新遞迴狀態，再透過 gain 機制（結合當前編碼與前態）調節訊息傳遞量。【各 feature 與算法與選擇方式】【**gain 機制運作方式？**】

最終輸出：bid/ask 兩維 action value 與下一狀態。【**判斷方法(模型怎麼做出輸出？一個機率嗎**】

我們額外測試了兩個生物啟發變體作為受控消融：Dale's Law（強制興奮/抑制神經元符號約束）與 STP-like 動態突觸增益；前者實測降低效能而未採用，後者（結合 cosine learning-rate schedule 穩定收斂後）成為最終 HLS 部署版本，顯示「加入生物機制」本身不保證有益，須逐一消融驗證。

對照組採用 TCN（5 個 residual block，dilation 1/2/4/8/16，僅用 ReLU 非線性，因果零填補避免未來資訊洩漏），選擇理由是它是委託簿時序建模的常見基準，且純前饋、可平行化的結構天然適合與遞迴的 PHC 做「是否需要遞迴」的對照實驗。【**加入消融實驗及其結果？**】

2.2 軟體驗證與回測方法

資料：為台指期 L2 逐筆委託簿事件，依交易日切分為 development（可調參）、used-oos-diagnostic（僅供觀察不可回頭調參）、final-holdout（終身只能開鎖一次）三類角色，避免資訊洩漏與隱性過擬合。

評估指標：為 causal-fill 序列化超額 PnL（僅使用因果可得資訊模擬撮合與庫存），而非單純的預測準確率，理由是 action-value 模型的最終價值在於交易決策品質，而非逐點預測誤差。

除主要模型比較外，另設計兩類消融：

(1) 蒸餾對照——以 PHC 為 teacher、TCN 為 student，比較 scratch 與 distilled 訓練的差異；

(2) 記憶機制反事實測試——將學到的 decay 替換為固定值、關閉記憶、或打亂序列順序，用以定位模型優勢的真正來源，而非僅報告單一最佳結果。

2.3 FPGA 部署方法與取捨

優先嘗試 hls4ml 自動轉換，但在 TCN 的 ConstantPad1d 層解析階段即失敗（連 C++ 專案都無法生成），且對入門級 FPGA（220 DSP 預算）而言，自動化工具的展開策略通常直接超出資源上限；因此改採手刻 HLS，以換取對每個 pragma 的精確控制與可除錯性，代價是工程時間成本更高。

部署流程遵循四級驗證鏈：C 模擬（csim）確認功能正確性、C 綜合（csynth）取得資源與延遲估計、RTL 協同模擬（cosim）確認 RTL 與 C 模型行為一致、最終 Vivado 布局布線（P&R）取得時序收斂與實測資源數字——本報告的每個關鍵數字都標註其達到的驗證等級，避免用 pre-synthesis 估計值冒充實測結果。

資源優化以八輪迭代收斂，核心取捨是「延遲 vs 資源」：關閉 PIPELINE（HLS PIPELINE off pragma）大幅降低資源用量，但代價是遞迴模型的下一時刻依賴當前狀態輸出，有效 II 等於 latency，無法透過 pipelining 提升吞吐——此取捨在入門級板子的資源預算下是必要的，但也是後續要逼近 1μs 級延遲時必須重新面對的架構限制。

---

3. Experiments

寫作原則：把密集數字放進表格、不放進段落；段落只負責一句話結論 + 驗證等級。這樣同樣的資訊量可以省下至少一半的字數。

3.1 資料與設定

台指期 L2 事件共 14,256,951 筆【幾天？】，切分為 6 天 train、1 天 val、2 天 test、4 天 NEW4（診斷用）、以及尚未開鎖的 NEW5 holdout。PHC-lite（hidden_dim=16，1,731 參數）與 TCN 對照組（7,874 參數，receptive field 125）皆使用 16 維 causal-only 特徵、序列長度 128。

3.2 軟體結果

Causal-fill 超額 PnL（val / NEW3 / NEW4，驗證等級：回測，非上線實測）：

Policy	val	NEW3	NEW4
Simple L2 rule	103,260	303,700	72,415
HGB (ML baseline)	142,435	411,090	97,680
PHC-lite v1	170,310	488,870	121,765
PHC-STP（HLS 部署版）	171,835	611,095	129,160
TCN scratch	172,195	565,260	126,915
TCN distilled	173,650	587,795	133,720

排名 PHC/TCN ＞ HGB ＞ simple rule 在 9 組敏感度組合（3 種庫存上限 × 3 種撮合模型）下 100% 一致。TCN（scratch 與 distilled）在三個 split 上皆優於 PHC teacher，但僅 3 個 split 樣本，符號檢定無法拒絕「中位數相等」，不宜宣稱統計顯著。

消融（記憶機制反事實，val causal-fill PnL）：固定 decay=0.9 幾乎打平可學習 decay（Δ 約+470）；關閉記憶明顯轉弱但仍為正（Δ 約-803,235）；反轉/打亂序列順序轉負（Δ 約-837,330 / -642,420）。結論：優勢主要來自序列順序與閘控狀態結構本身，而非某個學到的具體 decay 值——這也是 NEW5 尚未開鎖的三個理由之一。

3.3 硬體結果

PHC-lite 資源優化八輪迭代（DSP 用量，csynth 估計，皆已 cosim 通過）：

版本	關鍵變更	DSP	LUT	FF	BRAM
v1	初版（雙精度 tanh）	2636%	2073%	441%	—
v2	65 點 LUT 定點 tanh	1603%	839%	441%	—
v6	移除 PIPELINE pragma	247%	113%	86%	34%
v7	移除 helper ARRAY_PARTITION	121%	77%	52%	0%
v8（採用版）	顯式 PIPELINE off	21%	46%	13%	2%

v8（seed2）P&R 實測：DSP 21.36%（47/220）、LUT 14.20%（7,555/53,200，較 csynth 估計再降 70%）、FF 7.72%、BRAM 2.14%；延遲 8,316 cycles ≈ 83.16µs @ 100MHz；時序 WNS=+0.020ns、TNS=0.000ns，22,941 個 endpoint 全數收斂。功能驗證：25,600 筆測試向量，csim/cosim 零錯配，最大絕對誤差 0.00725275；bid/ask 方向一致率 99.9375%（16 個不一致案例皆發生在 |ref|<0.00054 的近零方向時刻）。

FPGA vs CPU（PyTorch float32, no_grad, 10,000 次計時；驗證等級：FPGA=P&R 實測固定值，CPU=軟體實測）：

指標	FPGA (v8)	CPU (PyTorch)	備註
Latency mean	83.16 µs	117.5 µs	FPGA 快 1.4×
Latency median	83.16 µs	84.2 µs	接近
Latency p99	83.16 µs	778.8 µs	FPGA 快 9.4×
Jitter	0（固定）	高（GC/排程）	FPGA 核心優勢

誠實揭露：此 CPU 基準為 Python/PyTorch 版本（非 LibTorch C++/TorchScript），p99 中有一部分是 Python 層開銷，換用 C++ 基準會是更硬的對照，列為 future work。此外，TCN 手刻 HLS（07b）目前僅完成 csim/csynth，尚未跑 cosim 與 P&R，其延遲數字仍是 pre-synthesis 估計，不可與 PHC 的 P&R 實測值直接類比。

3.4 邊界與限制（誠實揭露，建議放 Discussion 呼應）

- 資料時間戳最小解析度為 1ms，而延遲數字（23.16/83.16µs）遠低於此解析度，因此無法直接用本資料集驗證「次毫秒級延遲差異」對 PnL 的因果效應；延遲注入敏感度掃描顯示在 10ms 以下多數情況無顯著 PnL 退化——這是資料解析度的限制，不是「FPGA 延遲不重要」的證明。

- PYNQ-Z2 實機驗證僅完成 wrapper 介面 preflight，尚未上板；HLS 全流程僅完成 seed2，seed0/1 尚未開始，跨種子穩定性未知。

已經到了 23us？更新這個部分